3/8/26

PPOO Paradigmas e programação O. a objetos

-> Linguagens imperativas / Procedimentais (Pyth, C)
-> Linguagens orientadas a objeto (Java, Kotlin, Dart, ect).
-> Linguagens lógicas (Prolog, etc)
-> Linguagens funcionais (Lisp, Clojure)
-> Linguagens Declarativas (SQL, etc)

POO é programação orientada a peças, objetos são peças de software

1º trabalho -> produzir 1 caderno físico, manuscrito, organizado e completo com conteúdo dado/or, anteriores e em ordem cronológica

Instrumento de avaliação
2 provas P1 e P2
1 prova REC
3 notas práticas T_1, T_2 e T_3

Critérios de avaliação
$M_{TEO} = (P_1 + 2 \times P_2) / 3$
Se $M_{TEO} < 5$ $M_{TEO} = (M_{TEO} + REC) / 2$
$M_{PRAT} = (T_1 + 2T_2 + 2T_3) / 5$
Se $M_{TEO} \geq 5$ e $M_{PRAT} \geq 5$
$M_{FINAL} = 0,6 M_{TEO} + 0,3 M_{PRAT} + 0,1 F_i$
Senão
M_{FINAL} será M_{TEO} antes de M_{PRAT}, a menor entre as duas

CADERNO INTELIGENTE

Scanned with CamScanner

05/8/26

Introdução

A maioria dos produtos feitos pela humanidade é modular, mas há uns poucos que são monolíticos

1. Conceitos fundamentais de POO

1.1 - Classe

- representam tipos de peças de software que poderão ser usadas em programas. Se caracterizam por possuir nome, estrutura interna (atributos) e comportamento (métodos, que podem ser procedimentos ou funções.
- atributos são características de uma peça de SW, podem ser constantes ou variáveis, podem ser publicos, privativos ou protegidos, podem ser únicos (um só sem importar quantas peças daquele tipo existem) ou multiplos (um para cada peça daquele tipo que existir)
- atributos constantes representam características imutáveis, como RA, pai e mãe em uma classe aluno
- atributos variáveis representam características alteráveis como endereço e telefone em uma classe aluno
- atributos privativos são inacessíveis e inalteráveis fora da classe onde foram definidos. Atributos publicos são o contrário.
- atributos protegidos serão estudados quando estudarmos herança.
- atributos únicos existem uma única vez e estão associados a classe, já os atributos com multiplicidade estão associados às peças de SW que criarmos, para criar um atributo único basta

CADERNO INTELIGENTE

Scanned with CamScanner

5/8/16

usar a palavra static em sua declaração. Para tornar constante um atributo, basta usar a palavra final em sua declaração.

* atributo variável não pode ser público *

Desgraçadamente, às vezes é preciso que programas alterem valores de atributos variáveis, como por exemplo mudar o Telefone de um aluno, neste caso, implementaremos métodos alteradores de valor que recebem como parâmetro o novo valor do atributo, valida e, sendo válido, altera o valor do atributo. Tais métodos são chamados de SETTERS e seus nomes devem começar com a palavra SET e seguir com o nome do atributo.

Às vezes, é preciso recuperar o valor de um atributo variável, para por exemplo, escrever na tela. Para isso implementaremos um método recuperador cuja missão é retornar o valor do atributo. Tais métodos são chamados de GETTERS e seus nomes devem começar com a palavra GET e seguir com o nome do atributo, mas se o getter retornar um boolean, ao invés de GET usamos a palavra IS.

Tudo é possível: ter getter, mas não ter setter. Ter setter mas não ter getter, não ter nenhum ou ter os dois. Precisamos julgar o que é cabível caso a caso.

CADERNO INTELIGENTE

Scanned with CamScanner

10/8/26

Métodos

Representam o comportamento esperado de peças de SW que tem a classe como tipo. Podem ser procedimentos ou funções (procedimentos não tem retorno (VOID) ou função (tem retorno e não são VOID).

Quando representam serviços completos que as peças daquela classe prestam ao programa, devem ser públicos.

Quando representam parte de um serviço não podem ser públicos.

A presença da palavra `static` em sua declaração faz com que possam ser usados ainda que não existam nenhuma peça da classe de SW daquele tipo; neste caso são acionados a partir da própria classe. Na ausência da palavra `static`, para serem executados, precisam que exista uma peça de SW daquele tipo para atuar como acionador do método e, dentro do método, referir-se ao acionador usando-se a palavra `this`.

Há um estranho método o qual chamamos de **construtor** que tem como papel inicializar os atributos de uma peça de SW recém criada. São uma poderosa ferramenta antierro, pois impede que as peças sejam criadas com atributos que tem valores absurdos. Em muitas linguagens quando um atributo fica sem inicialização de alguma um valor imprevisível que por acaso estava na memória ocupada pelo atributo. Não em Java. Em Java, números são inicializados com zero, booleans ficam valendo false, chars ficam valendo o caractere cujo código é zero, objetos e vetores, que são ponteiros ficam valendo `null`. Apesar de

CADERNO INTELIGENTE

Scanned with CamScanner

previsível, esses valores ainda são considerados lixo.

Construtores tem o papel de inicializar atributos e setters de atualizar atributos e esses papéis não devem ser misturados.

Entre as especificidades dos construtores, podemos citar:

a) não podem escolher livremente seus nomes, que precisam necessariamente ser o mesmo da classe.

b) Não retornam nada e não são void, embora possa esperado que sejam declarados como, mas usar void ou qualquer outro tipo de indicação de tipo de retorno dá erro de compilação no contrutor.

c) Nunca são chamados explicitamente, isso faz e simplesmente declarados no programa para que entrem em execução espontânea quando uma peça do SW for criada. Quando uma classe não tem contrutor, peças de SW são criadas usando o comando NEW; criamos uma peça de uma classe chamada C escrevendo NEW C(); se isso fosse com que os atributos ficassem sem inicialização, o que seria uma porquice.

Fazendo contrutores numa classe C, passaríamos a ser obrigados a escrever New C(param1, param2, ... paramN) sendo cada um dos params um dos parametros do contrutor. Como qualquer outro método, pode ser declarado mais de uma vez com o mesmo nome e isso chamamos de sobrecarga. Decide-se qual método executar com base nos parâmetros passados. Os tipos dos parâmetros na ordem em que foram usados para definir um método funcionam como uma espécie de

CADERNO INTELIGENTE

Scanned with CamScanner

10/8/26

Sobrenome é o nome junto com os tipos seria o nome completo, o qual chamamos de assinatura.
Todo programa deve ter uma classe cujo nome podemos escolher, mas que habitualmente chamamos de MAIN. Esta classe deve ter um método com este cabeçalho:

public static void main(String[] args)

é é nele que a execução do programa começa.
Essa classe não deveria ter mais nenhum método.

17/8/26

1.2 Instâncias

Elas representam verdadeiramente peças de SW e, para ser justo, POO seria melhor nomeado se se chamasse PO a instâncias (POi) já que tudo gira em torno delas e elas determinam tudo, são as estrelas do show.
Elas seguem tudo o que foi previsto em sua classe e possui todos os atributos e todos os métodos declarados em sua classe que funcionará como uma espécie de molde para as instâncias.
São construídas pelo comando NEW, que é uma espécie de malloc capaz de reservar memória para os atributos das instâncias e que retorna o endereço onde foi/ser sido alocada na memória. Adicionalmente, rodam Construtores, coisa que não existe no tradicional malloc.
Em C, temos e precisamos usar o comando FREE, em Java isto é desnecessário existe um mecanismo chamado GARBAGE COLLECTION que libera toda a memória que não for mais acessível. O comando System.gc(); comanda manualmente a liberação de memória

CADERNO INTELIGENTE

Scanned with CamScanner

17/8/26

mas só costumamos usá-lo em situação onde haja restrição severas de memória; em outras deixamos no automático.

1.3 Objetos

Têm um papel secundário em relação a POO, servindo apenas para dar acesso às instâncias.

Com Data D1; declaramos um objeto da classe data. Ele não é mais que um ponteiro sendo capaz de guardar endereço de instâncias que criamos com NEW. Como ele no momento não serve para nada e usar d1.setDia((byte)19); daria erro de execução. Isso porque o método SetDia assim como qualquer método seria utilizado a partir de de uma instância o que ali não existe.

O comando (new Data((byte)19,(byte)1,(short)1965).setAno((short)1966); funciona, mas a instância criada ali com o NEW só será usada uma única vez, no caso, para ajustar o ano. Nunca mais poderá ser utilizada, pois não há acesso a ela.

Já os comandos abaixo:

```
Data d1=(new Data((byte)19,(byte)1,(short)1965);
    d1.setAno((short)1966);
System.out.println(d1.getDia()+"/"+d1.getMes()+"/"+d1.getAno());
```

acessam a instância criada com NEW 4 vezes, uma para ajustar o ano e as demais para recuperar dia, mês e ano para que seja printada.

Nos comandos abaixo

CADERNO INTELIGENTE

Scanned with CamScanner

17/8/26

Nos comandos abaixo:

```
Data d1 = new Data((byte)19, (byte)1, (short)1965);
Data d2 = new Data((byte)19, (byte)1, (short)1966);
Data d3 = d1;

System.out.println(d1.getDia()+"/"+d1.getMes()+"/"+d1.getAno());
//printa 19/1/1965
  d2.setAno(1966);
System.out.println(d1.getDia()+"/"+d1.getMes()+"/"+d2.getAno());
//printa 19/1/1965
  d3.setAno((short)1966);
System.out.println(d1.getDia()+"/"+d1.getMes()+"/"+d1.getAno());
//printa 19/1/1966
```

d3 possui um ponteiro que aponta para o mesmo endereço de d1

2. Métodos de implementação obrigatória

Existem em Java uma classe chamada **OBJECT** e dela todas as classes nativas ou feitas por nós herdam estrutura e comportamento. Herdar significa ter sem precisar implementar.

Há vários métodos herdados que estão OK, mas 3 deles vêm para nos novas classes com problemas de funcionamento e cabe a nós consertar. São eles **equals**, **hashCode** e **toString**.

toString tem o papel de gerar uma representação textual de uma instância.

Equals tem o papel de comparar duas

CADERNO INTELIGENTE

Scanned with CamScanner

instâncias em termos de atributos (== e != comparam em termos de endereços de memória, comparam ponteiros)

HashCode tem o papel de gerar um código que tem o papel de propiciar o armazenamento e a recuperação de instâncias de forma eficiente em estruturas de alto desempenho (o que pode consumir memória).

Devem começar com @Override.

Classes que não possuem nenhum atributo static ou que não tem nenhum atributo são ditas classes não replicáveis e isto quer dizer que não faz sentido criar instâncias daquela classe já que se o fizessemos estaríamos criando instâncias ocas.

Classes não replicáveis não tem os 3 métodos obrigatórios (estudados e que estudaremos) já que esses métodos se aplicam a instâncias e usam em suas lógicas os atributos não static que existirem.

31/8/26 Herança de Comparable

Trata-se de uma interface que existe em Java do qual devem herdar classes cujas instâncias podem ser comparadas de forma a determinar se uma é maior do que a outra forma a determinar se uma é maior que a outra.

Ver se duas instâncias são iguais ou diferentes é algo que sempre dá para fazer usando equals, herdado compulsoriamente de Object. Detectar que uma instância é maior ou menor que outra nem sempre é possível, isso fica reservado aos números,

CADERNO INTELIGENTE

Scanned with CamScanner

31/8/26

caracteres, strings e umas poucas classes como Data e Horário.

Interfaces são como classes que tem métodos abstratos. Método abstrato é um método que tem cabeçalho com nome, parâmetros e retorno, mas não tem uma lógica de programação que define seu funcionamento. Servem para serem implementadas pela classe que herdá-los.

É o que acontece com Comparable que tem o método compareTo que deve comparar a instância chamante do método (THIS) com o seu parâmetro que necessariamente será do tipo Object digamos obj. Caso THIS seja maior que obj, o método deverá retornar um inteiro positivo. Caso this seja menor que obj deverá retornar um inteiro negativo. Se this for igual, deverá retornar 0. Comparable e equals sempre devem concordar um com o outro.

No cabeçalho da classe que implementar Comparable deve estar escrito implements Comparable e na frente entre angle brackets (< e >) deverá estar escrito o nome da classe.

Não se sabe exatamente qual é o valor negativo ou o positivo retornado e, portanto deve-se sempre comparar com 0 o retorno, vendo se é menor, se é maior ou se é igual a zero.

Quando classes implementam interfaces expressamos isto com a palavra implements, mas quando classes herdam de classes ou interfaces herdam de interfaces, a palavra que se utiliza é extends. Classes podem herdar somente de uma única classe, mas pode

CADERNO INTELIGENTE

Scanned with CamScanner

31/8/26

implementar basicas interfaces.

-Qual é o criterio para usar Comparable?

-Resumo sobre quando usara cada metodo disgutido!

Herança de Cloneable

Eventualmente é necessário fazer uma cópia de uma instância, provavelmente porque desejamos fazer alguma tipo de alteração na instância sem perder seus valores originais. A forma oficial e padronizada implementar chamado clone e, para implementá-lo usar um construtor de cópia que também implementaremos. Clone e construtor de cópia são dois metodos inseparáveis.

O que caracteriza um construtor de cópia é ter um unico parâmetro cujo tipo é a própria classe.

Ter dois metodos devem estar presentes em classes cujas instâncias são alteráveis. Setters são alteradores classicos, mas não são os unicos.

09/09/26 Os atributos são privados para garantir que outros não os usem de forma indevida, que não consigam alterá-los. Por isso, quando outros precisam alterar/acessar atributos fazemos getters e setters e eles simplesmente guardam ou retornam o que foi selecionado como parâmetro ou solicitado ao getter. Porém isso só é adequado se o atributo tiver um tipo primitivo ou

CADERNO INTELIGENTE

Scanned with CamScanner

9/9/26

um objeto de uma classe inalterável. Se for objeto de uma classe alterável, antes de guardar ou retornar, devemos clonar.

O simples armazenamento de ponteiro é chamado de SHALLOW COPY, cópia rasa; a geração de um novo ponteiro, de um clone, é chamada de DEEP COPY, cópia profunda.

Shallow Copies compartilham o mesmo ponteiro, Deep copies tem ponteiros diferentes.

14/9/26 Quando fazer ou não cada um dos seis métodos obrigatórios ou proibitivos

A primeira regra é existir na classe um ou mais atributos variáveis não static. A satisfação dessa regra é suficiente para decidir fazer os métodos toString, equals e hashCode. Para os demais, essa condição é necessária, mas não suficiente.

Para decidir fazer o método compareTo é necessário também conseguir encontrar um critério capaz de comparar duas instâncias de uma classe resultando a comparação em exatamente 1 de 3 possíveis resultados: ou a primeira instância é maior do que a segunda, ou a segunda é maior do que a primeira ou ambas são iguais, outras possibilidades não são admissíveis.

Para os métodos clone e construtor de cópia (que são inseparáveis, sempre ausentes ou presentes) deve-se verificar na classe há algum método

CADERNO INTELIGENTE

Scanned with CamScanner

14/9/26

que altere atributos do this. Setters são sempre o primeiro deles, mas na ausência deles, qualquer método que altere um ou mais atributos do this será, exceto construtores.

Classes Genéricas 21/9/26

Habitualmente apenas métodos recebem parâmetros. Classes genéricas são classes que têm parâmetros. Elas são particularmente úteis para implementar EDs pois os parâmetros delas, que são classes, podem indicar que tipo de coisa armazenar nas estruturas, permitindo assim a construção de estruturas cazes de guardar o que quer que desejemos.

Java não permite que instanciemos objetos e nem mesmo seja o parâmetro de uma classe genérica, por isso, em caso de necessidade usamos o truque de instanciar Object.

Não é permitido criar uma deep copy cujo tipo seja o parâmetro de uma classe genérica de um jeito simples, chamando clone ou usando um construtor de cópia, mas no caso de serem clonáveis podemos invocar o método clone não diretamente, mas sim usando a biblioteca Reflect.

Quando o tipo do parâmetro de um setter ou de retorno de um getter é criado, podemos decidir por uma simples inspeção à documentação de ele é ou não clonável. O mesmo não ocorre no

CADERNO INTELIGENTE

Scanned with CamScanner

21/9/26

tipo for parâmetro de uma classe genérica; Não sermos capazes de saber através de um simples exame ou consulta à documentação nada sobre sua clonabilidade e para decidir sobre isso precisaremos implementar alguns testes em nossos programas.
Assim o programa analisará o que entra em nossa classe genérica ou o que sai dela para decidir em Tempo de execução sobre a clonabilidade o que não é possível de ser feito em tempo anterior ao tempo de compilação.

05/10/26

Herança e Polimorfismo

Herança é uma relação que pode se estabelecer entre:

A) Uma classe e outra classe, feita neste caso com a palavra extends

B) Uma classe e uma ou mais interfaces, usa-se neste caso a palavra implements

C) Uma interface e uma ou mais interfaces, usa neste caso a palavra extends.

d) Um parâmetro de classe genérica e classes ou interfaces das quais pretendemos exigir a existência de herança, usa-se neste caso a palavra extends.

CADERNO INTELIGENTE

Scanned with CamScanner

05/10/26

Métodos abstratos são métodos que tem nome, tipo de retorno, parâmetros, especificação de encapsulamento e de lançamento de exceção, mas não tem associado a eles nenhuma lógica de programação. Servem para indicar a existência de um padrão de comportamento cuja implementação é deixada em aberto para ser feita posteriormente numa classe que os herda.

Numa situação de herança, nós temos a base (a classe ou interface da qual se herda) e a derivada (a classe ou interface que herdam da base).

Algo muito interessante acontece nas heranças: sempre que um objeto cujo tipo é a base é instanciado com instâncias cujo tipo for qualquer derivada direta ou indireta da base, ao usar o objeto para chamar um método:

a) o método tem que existir na classe base ou não poderá ser chamado.

b) o método que efetivamente será o primeiro que for encontrado a partir da classe da instância num caminho ascendente na hierarquia de herança sempre à base.

Interfaces somente podem ter métodos abstratos e atributos constantes. No entanto, métodos abstratos podem eventualmente ser encontrados em classes que, neste caso, são consideradas classes abstratas, e são identificadas pela palavra abstract antes da palavra class. Nelas, os métodos

CADERNO INTELIGENTE

Scanned with CamScanner

06/10/26

abstrata necessariamente são identificada pela palavra abstract após do tipo de retorno e do mesmo não precisa ocorrer em interfaces.

Sempre dá para fazer uma interface tornar-se uma classe abstrata, o que é uma profunda estupidez. Isso porque só se pode herdar de uma classe, mas se pode herdar de muitas interfaces e fazer de uma interface uma classe seria limitante.

Chamamos de polimorfismo quando tornamos um objeto e chamamos dele um método e num momento o método que vemos executar é um e em outro momento é outro, tudo porque no primeiro momento o objeto estava instanciado com uma instância de um tipo e no outro momento com uma instância de outro tipo; naturalmente os tipos das instâncias e do próprio objeto fazem parte de uma relação de herança conforme especificados nos itens A e B acima.

Abstração, encapsulamento, herança e polimorfismo são as 4 propriedades fundamentais da orientação à objetos. Lembremos que encapsulamento é o controle de acesso que fazemos quando usamos as palavras private, public ou protected e lembremos que abstração é usar um objeto sem se preocupar com a forma pela qual ele efetivamente funciona.

Aquilo que declaramos protected, quando herdado, é acessível pelo que herdou; isso não ocorre com private. Protected em JAVA tem um sério inconveniente: classes cujos arquivos estão gravados na mesma pasta podem acessar o que é protected das outras independentemente de

CADERNO INTELIGENTE

Scanned with CamScanner

06/10/26

herança; isso não ocorre em outras linguagens e é algo com que temos que lidar, por exemplo, colocando classes que tem protected numa pasta só delas.

São como classes que ao serem declaradas em vez de usarmos a palavra class usamos a palavra enum. Em vez de compulsoriamente herdarem de Object, compulsoriamente herdam de Enum. De Enum herdam vários metodos, entre eles o metodo name que tem funcionamento idêntico do também herdado toString (retornam o nome da enumeração ao qual se refere), ordinal que retorna a posição do item na enumeração, compareTo que compara os valores retornados pelo metodo ordinal aplicado a cada instância podendo com enumValues que é um método static que retorna um vetor com todos os valores de enumeração, além do metodo valueOf que retorna o valor da enumeração que corresponde ao String dado como parâmetro.

As instâncias de uma enumeração:

a) são criadas quando é declarada a enumeração e nenhuma outra pode ser criada posteriormente através e nenhuma outra pode ser criada posteriormente de new.

b) são necessariamente constantes.

c) podem ter atributos que também são necessariamente constantes.

d) podem ter métodos e eles não podem alterar atributos já que eles são constantes.

CADERNO INTELIGENTE

Scanned with CamScanner

e) podem ter construtores.

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

Aula assíncrona 1: Regras de Nomenclatura

Java é uma linguagem CASE SENSITIVE

↳ diferencia maiúsculas e minúsculas em nomes

int dia, Dia, DIa, diA, dIA, DiA, DIA, dIa;

↳ 8 variáveis diferentes

não é uma boa prática de programação

→ existem regras para que não haja confusão nas nomenclaturas

→ Classes tem nomes justapostos, quando possuem mais de uma palavra no nome (sem espaço ou underlines). A regra também diz que as letras iniciais de cada palavra devem ser maiúsculas e as demais todas minúsculas.

qualquer classe pertence mesma regra { InputStreamReader, BufferedReader, ObjectInputStream, Integer, String

public class data {...} X Não

public class Data {...} Sim

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

Classes e métodos que implementarmos não precisam obrigatoriamente as regras, não vai dar erro de compilação, mas isso gera confusão no uso do que fizermos, pois os bons desenvolvedores estão acostumados com as regras de nomenclatura.

Classes, métodos e atributos que não seguirem as regras de nomenclatura são feitos por pessoas incompetentes. Não seguir as regras dificulta o uso do programa. Para um programa ser bem usado, precisa ser bem feito e funcionar, as duas coisas. Não pode ter somente uma dessas coisas.

Regras:

I) As palavras que formam nome de **classes**, **interfaces** ou **projeto** devem estar em PascalCase. Isto é, justapostas, com iniciais maiúsculas e as demais minúsculas. ex: Programa, Data.

II) As palavras que formam o nome de um **pacote** (biblioteca) devem ser separadas por ponto (.) e escritas totalmente em minúsculo. ex: java.io, java.awt.event.

III) As palavras que formam nome de **variável** ou **método** seguem o camelCase, a inicial da primeira palavra é minúscula e as demais iniciais são maiúsculas. ex:

dia	setDia
mes	setMes
ano	setAno

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

IV) As palavras que formam o nome de uma **Constante** devem ser separadas por sublinhado (_) e escritas totalmente em maiúsculo. ex MAX_VALUE, MIN_VALUE.

V) A primeira palavra de um método que altera valores de um objeto deve ser "set" e o resto deve expressar o é alterado; é um **método setter**.

VI) A primeira palavra dom nome de um método que recupera valor de um objeto deve ser "get" e o resto deve expressar o que é alterado, é um **método Getter**.
OBS: se for boolean, usa is ao invés de get.

Aula Assincrona 2: Tipos vs Classes Wrapper (pt.1)

boxing e unboxing
boxed e unboxed

Tipos Primitivos do Java

tipos	Memória	Valores
byte	1 byte	de -128 a 127
short	2 bytes	de -32768 a 32767
int	4 bytes	de -2147483648 a 2147483647
long	8 bytes	de -9223372036854775808 a 9223372036854775807
boolean	1 byte	false e true
char	2 bytes	65536 caracteres diferentes (UNICODE)

256 ASCII = 256 1os UNICODE

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

TIPOS	MEMÓRIA	VALORES
float	4 bytes	de -3.40282347e+38 a -1.402398 46e-46 -0 e 0, de 1.402398 46e-46 a 3.402823 47e+38 valores especiais: NaN, -∞, +∞
double	8 bytes	de -1.7976931348623157 0e+308 a -4.9406564 584124654 4e-324, -0 e 0 de 4.9406564 584124654 4e-324 a 1.7976931348623157 0e+308, valores especiais: NaN, -∞, +∞

Classes wrapper

São classes que "embrulham" os tipos, isto é, dentro de um objeto da classe wrapper tem um tipo primitivo daquele (em forma de atributo).

CLASSES	TIPO
Byte	byte
Short	short
Integer	int
Long	long
Float	float
Double	double
Boolean	boolean
Character	char

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

→ Quando se tem um tipo guardado numa variável de tipo, dizemos que está **unboxed**
→ Quando um tipo é guardado num objeto de uma classe wrapper, diz-se que está **boxed**

```
int a=3, b=7, media;
media = (a+b)/2;
```

é o mesmo que

```
Integer a=3, b=7, media;
media = (a+b)/2;
```

e também é o mesmo que

```
Integer a = new Integer(3), b= new Integer(7), media;
media = new Integer((a.intValue() + b.intValue())/2);
```

Quando usar valores BOXED ou UNBOXED?

→ tipos primitivos ou int de classe gasta menos memória, leva menos tempo para executar
→ Classe Wrapper é mais ineficiente
→ Ainda sim, usar wrappers pode ser útil pois os métodos da classe são bem úteis e já podem fazer muitos processamentos automáticos. Tipos primitivos não possuem métodos.
→ Todas as classes Wrappers possuem métodos úteis.

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

Aula Assíncrona III: Tipos vs classes Wrappers (Pt.2)

Existe uma diferença entre usar `Integer a = new Integer(7);` e `Integer a = 7;`

Cenário 1:

```
Integer a = new Integer(7);
Integer b = new Integer(7);
Integer c = new Integer(7);

System.out.println(a == b); // false
System.out.println(a == c); // false
System.out.println(b == c); // false

System.out.println(a.equals(b)); // true
System.out.println(a.equals(c)); // true
System.out.println(b.equals(c)); // true
```

Cada uso de **new** gera uma nova instância, um novo endereço de memória onde o valor é guardado

== Sempre dá false porque cada um dos Integer está num endereço de memória diferente

equals dá true porque compara os valores, e apesar de serem 3 objetos, neste cenário tem o mesmo valor

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

Cenário 2:

```
Integer a = 7;
Integer b = 7;
Integer c = 7;

System.out.println(a == b); //true
System.out.println(a == c); //true
System.out.println(b == c); //true

System.out.println(a.equals(b)); //true
System.out.println(a.equals(c)); //true
System.out.println(b.equals(c)); //true
```

O primeiro uso, primeira atribuição equivale a new Integer(7).
Nas demais linhas não é criado um novo Integer, o que foi criado na linha 1 vai ter o endereço guardado nas demais
== dá true porque compartilham o mesmo endereço

Esta lógica serve para todas as classes Wrapper

CADERNO INTELIGENTE

Scanned with CamScanner

21/8/26

Cenário 3

```
String a = new String("Java");
String b = new String("Java");
String c = new String("Java");

System.out.println(a == b); //false
System.out.println(a == c); //false
System.out.println(b == c); //false

System.out.println(a.equals(b)); //true
System.out.println(a.equals(c)); //true
System.out.println(b.equals(c)); //true
```

Cria 3 instâncias, cada uma num endereço diferente

== dá false porque compara endereço, e cada um tem um endereço diferente.

equals compara valores, por isso dá True

→ Sem NEW é mais econômico em relação a memória

→ Ao não usar NEW, é melhor comparar com == para evitar gastar memória, pois ser mais eficiente só comparar endereço.

↳ Só vale para Wrappers e String

CADERNO INTELIGENTE

Scanned with CamScanner

22/8/26

Cenário 4:

```
String a = "Java";
String b = "Java";
String c = "Java";

System.out.println(a==b); //True
System.out.println(a==c); //True
System.out.println(b==c); //True

System.out.println(a.equals(b)); //true
System.out.println(a.equals(c)); //true
System.out.println(b.equals(c)); //true
```

O primeira vez que guardarem "Java" numa String vai guardar e gerar uma instância que vai guardar o endereço.

A segunda e terceira vez não cria uma nova instância, porque usa o endereço já criado na primeira instância e guarda em ambas.

Isso porque está guardando mais de uma vez em um objeto da classe String.

A primeira linha é equivalente a instanciar com NEW.

Por ter o mesmo endereço, pode-se comparar com equals ou com ==

CADERNO INTELIGENTE

Scanned with CamScanner

23/9/26

1. Servidor de Continhas 1: visão funcional

Usando Threads e Sockets para criar um sistema com arquitetura Cliente/Servidor

-> Um servidor é executado antes
-> O programa cliente é executado e ele se conecta ao servidor
-> O cliente pode avisar ao servidor
-> o servidor atende ao cliente
-> o cliente encerra sua conexão
-> o servidor pode ser desligado

Usando o Fazedor de Continhas

1) O servidor é executado, ele fica aguardando conexão de clientes; cada cliente digita "desativar", o servidor envia a todos os clientes conectados que o servidor será desconectado e então o servidor termina sua execução; os programas clientes informam ao usuário que o servidor está inoperante e pede para tentarem novamente

2) O programa cliente pode ser executado quantas vezes for desejado e, toda vez, ele vai se conectar ao servidor; lá no servidor existe uma variável double valendo 0 (zero); o cliente pode enviar solicitação para somar um valor dessa variável double, subtrair, multiplicar, dividir e todas essas operações fazem o servidor alterar o valor que foi associado àquele cliente; o cliente

CADERNO INTELIGENTE

Scanned with CamScanner

23/9/26

pode também solicitar e ler o valor atual desta variável, o servidor retornará este valor; o cliente pode também pedir para desconectar e o servidor desconecta aquele cliente

O Cliente funciona sem o servidor, mas nada acontece, dada a arquitetura a funcionalidade só está ok quando ambos estão sendo executados.

2. Fazedor de Continhas 2: Classes comuns ao Cliente e ao Servidor

- São 6 classes cujos objetos são transmitidos do cliente para o servidor ou vice-versa

- Comunicado
  - topo da hierarquia, todas as demais herdam dela
  - é uma classe de java.io.*
  - implementa Serializable, é o que precisa para poder ser transmitido entre programas via socket (tem outros usos também, como gravação em BD)

- PedidoDeOperacao
  - herda de Comunicado
  - Construtor recebe a operação matemática e também o valor recebido (para ser o segundo operando junto com o valor que existe no servidor).
  - Usa getters

CADERNO INTELIGENTE

Scanned with CamScanner

13/9/26

→ PedidoDeResultado
- classe oca, filha de Comunicado
- sempre que o cliente quiser saber o seu resultado ele manda um objeto desta classe e o servidor responde com um objeto da própria classe

→ Resultado
- filha de Comunicado
- tem o valor resultante como atributo

→ PedidoParaSair
- Classe oca, filha de Comunicado
- quando o usuário errar ~~desistir~~, o cliente manda um objeto desta classe e o servidor recebe, fazendo o mesmo

→ ComunicadoDeDesligamento
- classe oca, filha de Comunicado
- quando alguém desativar no servidor, ele manda objetos desta classe para todos os clientes conectados

Outras classes comuns que não se relacionam à arquitetura cliente/servidor:

Teclado → classe usada para gerenciar a digitação de dados pelo usuário

Parceiro
- no cliente representa o servidor
- no servidor representa o cliente

CADERNO INTELIGENTE

Scanned with CamScanner

23/9/26

Parceiro
- tem um Socket
- tem um objectInputStream e objectOutput Stream (receptor e transmissor)
- usa Comunicado
- tem um Semaphore Mutex (mútua exclusão) Como usa 1 recurso: quando o solicitante usar, ele para e só retoma quando o recurso estiver disponível

- O Construtor usa o Socket, o objectInputStream e o objectOutputStream

Métodos
- recebe (Comunicado) } todos retornam um
- espie () } Comunicado
- envie ()

→ faz o servidor ou o cliente receber um Comunicado (envia)
→ olha o que foi enviado sem consumir
→ faz o parceiro enviar ou (receber) que foi enviado

→ espie = Sempre recebe a mesma coisa (não consome)
→ envie = Sempre recebe diferente (consome)

- O uso de thread no espie com Semaphore garante que não haja um "atraplho" e mistura, pois usa um recurso

Método adeus encerra tudo

CADERNO INTELIGENTE

Scanned with CamScanner

24/9/16

3. Fazendo de Conhinhas 3: implementação das classes específicas do cliente (parte 1)

Ações de iniciação que devem ser executadas no Cliente

1) Checar parâmetros recebidos na main pelo vetor args, para permitir que o usuário indique a máquina onde roda o Servidor e/ou a porta adotada pelo Servidor, ao qual o cliente deve se conectar, sinalizando eventuais erros, ou, caso nenhum parâmetro chegue em args, assumir que o Servidor roda em uma máquina padrão e que ele adotou uma máquina padrão.

2) Realizar a conexão com o Servidor, iniciando um transmissor (ObjectOutputStream) e um receptor (ObjectInputStream) para, com tudo isso, instanciar um Parceiro Servidor, que será usado para se comunicar com o Servidor.

3) "startar" uma TratadoraDeComunicadoDeDesligamento (THREAD) naturalmente compartilhando com ela o Parceiro Servidor de onde receberá, eventualmente, um ComunicadoDeDesligamento.

{ 1, 2 e 3 executadas no início da main do Cliente

CADERNO INTELIGENTE

Scanned with CamScanner

24/9/26

[após as ações 1, 2 e 3 no início, a main entra num loop (que é executado junto com a THREAD do item 3)]

Tipos de tarefas simultâneas que devem ser executadas no cliente após a realização das ações de início:

1) Continuamente (loop) interagir com o usuário, mostrando as opções que ele tem, solicitando um valor que funcionará como um segundo operando, caso ele tenha optado por fazer um PedidoDeOperacao, ou mostrando um Resultado, caso ele tenha optado por fazer um PedidoDeResultado, e sempre sinalizando erros por mau uso.

2) Continuamente (loop) aguardar um eventual ComunicadoDeDesligamento vindo do Servidor, para então tomar as medidas necessárias.

↳ Realizado pela THREAD tratadoraDeComunicadoDeDesligamento

CADERNO INTELIGENTE

Scanned with CamScanner

24/9/26

4. Fazedor de Continhas: Implementação das classes específicas do cliente (parte 2)

Sobre execução de programas

→ usuários não rodam em IDE's, prompt de comando é mais adequado (terminal, CLI)

→ CD: change directory, navega entre pastas no CLI

→ após ter o código compilado, rodar com JAVA nome Classe

→ informações passadas no CLI são lidas na main e recebidas no vetor args (String[] args)

ex: JAVA Servidor 7000

↳ Servidor usa essa porta

→ se rodar o Servidor com uma porta, precisa passar essa mesma porta para o Cliente

ex: JAVA Cliente localhost 7000

↳ ou IP

↳ mesma do Servidor

{ localhost é recebido em args no índice 0
{ 7000 é recebido em args no índice 1

neste programa, args espera 2 itens no máximo

Os itens recebidos em args são usados para instanciar um SOCKET e se args estiver vazio, existem dois ~~defaults~~ Contatos que são usados.

↳ Especificação do item 1 das ações executadas no início da main

CADERNO INTELIGENTE

Scanned with CamScanner

24/9/26

- instancia conexao
- com conexao.getOutputStream(); instancia o transmissor (objeto de ObjectOutputStream)
- com conexao.getInputStream(); instancia o receptor (objeto de ObjectInputStream)

- declara ParceiroServidor
- instancia com os 3: conexao, transmissor e receptor

→ Parte 2 das ações no início da main

- instancia a TratadoraDeComunicadoDeDesligamento com o ParceiroServidor
- começa a execução da THREAD com .start()

→ Parte 3 das ações no início da main

- tem um loop com do que tem um print das opções do usuário
- usa a classe Teclado para receber o que foi digitado
- se digitou algo inválido lança exceção e dá um continue (não executa nada do que vem depois e volta para o começo do loop)
- usa indexOf para validar se digitou uma das opções possíveis, se não printa erro e dá continue
- se a opção tiver sido válida, passa a pedir o valor e pede um double com a classe Teclado
- se digitar algo que não é número, lança exceção e dá um continue

CADERNO INTELIGENTE

Scanned with CamScanner

24/9/26

- válida se a pessoa tentou dividir por 0 e se sim, dá uma mensagem de erro e dá o continue
- vai instanciar um objeto de PedidoDeOperacao com a opção e o valor selecionados
- o objeto instanciado na linha de cima é passado para o servidor com servidor.receba

- pode ser que a pessoa selecionou "=", aí cai num else e instancia um objeto de PedidoDeResultado
- declara o Comunicado e num loop fica validando se o Comunicado recebido é um Resultado:

```
Comunicado comunicado = null;
do {
    comunicado = (Comunicado) servidor.espiel();
}
while (!(comunicado instanceof Resultado));
```

- Se passar disso, chama servidor.envie() e recebe o Resultado
- printa o resultado

- Se a pessoa escolher a opção "T", sai do loop
- saindo do loop chama o seguinte:

```
servidor.receba(new pedidoParaSair());
```

- com isso, esse cliente será desconectado do servidor

- no fim executa System.exit(0);
- exit garante que a THREAD finaliza junto com o programa

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

5. Fazedor de Continuas 5: Implementação das classes específicas do Servidor (parte 1)

Ação de iniciação que deve ser executada no Servidor:

1) Checar os parâmetros vindos para a main em args para permitir que o usuário indique a porta adotada para aceitar conexão dos Clientes, sinalizando eventuais erros, ou, caso nenhum parâmetro venha, adotar uma porta padrão.

2) Declarar e instanciar uma ED (ArrayList) "usuarios" para armazenar objetos da classe ParceiroDoServidor onde estarão todos os acessos dos clientes conectados (que são os parceiros do Servidor); tais acessos serão usados ao longo de todos os programas para se comunicar com os clientes.

3) "Startar" uma AceitadoraDeConexao (THREAD), fornecendo a ela a porta na qual conexões serão aceitas e, naturalmente, compartilhando com ela a ED usuarios que ela compartilhará com cada SupervisoraDeConexao (THREAD) que ela Startar, por ocasião de cada conexão que ela aceitar.

Lt 1, 2 e 3 são feitas no início da main

args deve ter 0 ou 1 item

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

ATALHOS

Anteriormente foi visto sobre como rodar um programa Java via CMD, mas existe alternativa, que é a criação de atalhos que ao serem executados, rodam os programas.

Para criar um atalho é simples:

→ abre onde o atalho será criado (ex.: área de trabalho)

→ selecionar Novo → atalho

→ na janela que abrir, incluir o comando que será executado no CMD e depois dar um nome à ele

→ quando o atalho for criado, clicar com o botão direito e abrir propriedades

→ no campo "destino", incluir o caminho do JAVA no PC

→ no campo "iniciar em", incluir a partir onde todos os compilados do programa estão (alterar o caminho)

→ a partir daí o atalho funciona

Toda cada conexão aceita na main, uma nova SupervisoraDeConexao é instanciada

AceitadoraDeConexao 1:N SupervisoraDeConexao

Após a execução das ações de iniciação no início da main, o programa passa a executar algumas ações simultâneas no Servidor

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

Tipos de tarefas simultâneas que devem ser executadas no Servidor após as tarefas de iniciação:

1) Continuamente interagir com o usuário adm, avisando que o servidor está ativo, informando-o que pode desativar o servidor, aguardando que algo seja digitado, informado que o que foi digitado está incorreto caso esteja de fato, ou caso tenha sido digitado "desativar", enviando a todos os Parceiros cliente encadeados uma Comunicado de Desligamento.

↳ isso executa no final da main junto com as THREADS iniciadas no início do programa

2) Continuamente aceitar novas conexão com os programas cliente que pedirem, ou seja, "startar" uma nova SupervisoraDeConexao para supervisioná-la, ou seja, atender qualquer solicitação recebida via a conexão e naturalmente compartilhada a ED de usuários.

↳ realizado pela THREAD AceitadoraDeConexao

3) Inicialmente, instancia um transmissor (objectOutputStream) e um receptor (objectInputStream) associados à Conexao que ela supervisiona, para com isso, instanciar um ParceiroCliente, que deverá usado ao longo de toda a sua execução para se comunicar com o programa Cliente, além de incluí-lo na ED usuários (para

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

aquele Parceiro cliente, como qualquer outro presente na ED, passa receber Comunicados do Servidor destinados à todos clientes conectados, o que acontece quando o Servidor recebe o comando "dusatinar"); após tudo isso, passa a receber continuamente o que for solicitado e enviando um Resultado ao cliente solicitante. no caso da solicitação de ser do tipo PedidoDeResultado, ou removendo seu Parceiro cliente da ED usuários e terminando sua execução, no caso da solicitação ser do tipo PedidoParaSair.

↳ executado por cada THREAD SupervisoraDeConexao

Parceiro cliente = new Parceiro(conexao, transmissor, receptor);

ArrayList usuarios = guarda todos os clientes conectados

→ tem o método adeus que desfaz toda a conexão e garante que as threads não executem mais

↳ é executado no final, quando recebe um PedidoParaSair

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

6. Fazedor de Continhas 6: Implementação das classes específicas do Servidor (parte 2)

- valida itens em args, pode ter 0 ou 1, se for mais, printa um erro e finaliza a execução
- se args tiver algo, isto passa a ser a porta em que vai escutar
- AceitadoraDeConexao é instanciada com a porta e o ArrayList usuarios, esta é uma THREAD
- se a porta passada no args for inválida, lança uma exceção e finaliza o programa

- tem um for(;;) que tem os prints das informações do servidor
- o servidor usa a classe Teclado para receber o que foi digitado e guarda numa String
- valida com o equals se o que foi digitado é o "desativar"
- usa synchronized(usuarios) para garantir que nenhuma outra thread use esta ED ao mesmo tempo
- tem um for each que manda um ComunicadoDesligamento para cada parceiro usuario na ED e escuta o método adeus de cada um deles

```
synchronized (usuarios) {
ComunicadoDesligamento comuni = new ComunicadoDesligamento();
for (Parceiro usuario : usuarios) {
    usuario.receba(comunicado);
    usuario.adeus();
}
```

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

Correção do texto do item 2 da seção "tipos de tarefas simultâneas que devem ser realizadas após a ação de iniciação":

2) Inicialmente, declara e instancia um ServerSocket, que é o que se usa para aceitar pedidos de conexão (tem um método accept), além de guardar em um atributo (para poder ser usado em todos os métodos da classe) a ED usuários que foi compartilhada pela main ao instanciar esta thread; depois, passa a continuamente aceitar novas conexões com os clientes que pedirem, declarando, instanciando e "startando" uma SupervisoraDeConexao para supervisionar, ou seja, atender as solicitações recebidas via conexão supervisionada e, naturalmente compartilhando com ela a ED "usuarios".

Classe AceitadoraDeConexao

- importa java.net.* porque é onde tem Socket
- importa java.util.* porque é onde tem ArrayList
- ferda de THREAD
- usa um ServerSocket pedido que é instanciado no contrutor com a porta (pode lançar exceção)
- tem uma ED usuarios
- por ser THREAD, tem o método run()
- no método run, declara um Socket

Socket conexao = THIS.pedido.accept();

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

- declara uma SupervisoraDeConexao e instancia com a conexão dos usuarios.
- a conexão é a mesma que foi aceitada antes, e com isso chama o metodo start() da supervisora

Classe SupervisoraDeConexao

- herda de THREAD
- importa java.io.* porque é onde tem ObjectInputStream e ObjectOutputStream
- importa java.util.* e java.net.*

- tem 4 atributos
  - double valor
  - Parceiro usuario
  - Socket conexao
  - ArrayList<Parceiro> usuarios

- tem o metodo run()
- instancia o transmissor e receptor associados a conexão que é atributo e que foi iniciada no construtor (recebeu da main)
- instancia o Parceiro

this.usuario = new Parceiro(conexao, receptor, transmissor);

- requisita uso exclusivo à ED usuarios com synchronized para adicionar mais o novo parceiro usuario

```
synchronized(this.usuarios){
    this.usuarios.add(this.usuario);
}
```

CADERNO INTELIGENTE

Scanned with CamScanner

25/9/26

- tem um for(;;)
- no for queda um comunicado que o usuário enviar
- se não veio comunicado, dá um return
- if(Comunicado instanceof PedidoDeOperacao)
  - declara um novo pedido operação a partir do comunicado
  - faz um switch para validar a operação e executá-la no atributo valor
- também valida se comunicado é instância de pedido de resultado
  - se for, instancia um Resultado e chama o Usuario.~~receba~~envie(), fazendo-o enviar
- Se o comunicado for PedidoParaSair

```
synchronized (this.usuarios) {
    this.usuarios.remove(this.usuario);
}
this.usuario.adeus();
```

- finaliza o for e a THREAD!

CADERNO INTELIGENTE

Scanned with CamScanner